

Warm-Hello

IT Operations Documentation

Architecture · Deployment · Configuration · Database · Monitoring · Security · Troubleshooting · Escalation

Version: Initial 1.0 release

Generated: Thursday, 03 September 2026, 18:17 UTC

Classification: Internal — Authorized personnel only

Table of Contents

1 Architecture & Infrastructure Map

- 1.1 System Diagram (Text)
- 1.2 Hosting & Resources
- 1.3 Network Topology
- 1.4 Horizontal Scaling Caveat (Today = NOT SUPPORTED)

2 Deployment & CI/CD Pipeline

- 2.1 Build Process
- 2.2 Environment Provisioning (Fresh Lightsail VM from scratch)
- 2.3 Standard Production Deployment Workflow (git push → SSH deploy)
- 2.4 Staging vs. Production (paths today + recommendation)
- 2.5 Rollback Strategy
- 2.6 Future Recommended CI/CD — GitHub Actions Starter

3 Configuration & Environment Variables

- 3.1 Secrets Management — Today
- 3.2 Environment Matrix (Complete Zod Key List)
- 3.3 Production `.env` file — minimum required block
- 3.4 Local Dev `.env` (Developer Laptop)
- 3.5 Changing an environment variable — live rollout procedure

4 Database & Data Operations

- 4.1 Engine Specs
- 4.2 Migration Procedures
- 4.3 Backup & Restore
- 4.4 Periodic DB Ops Manual Checklist

5 Monitoring, Logging & Observability

- 5.1 Health Checks
- 5.2 Log Aggregation & Rotation
- 5.3 Alerting Thresholds
- 5.4 Performance dashboards
- 5.5 Baseline numbers (reference for anomaly detection)

6 Security, Access & Compliance

- 6.1 Access Control
- 6.2 SSL/TLS & Domain Setup
- 6.3 Compliance & Data Privacy

7 Troubleshooting Runbooks

- 7.1 Runbook 1 — Next Build OOM SIGABRT on Deploy (exit code 134, 512 MB micro)
- 7.2 Runbook 2 — PM2 started, curl to `/api/health` fails "curl(7) Failed to connect: Connection refused" for first minute
- 7.3 Runbook 3 — App crash-loop immediately after pm2 start (Zod env parse fails)
- 7.4 Runbook 4 — Prisma "P1001 Can't reach database server at X.supabase.com:5432"

- 7.5 Runbook 5 — Too Many 429s on endpoints (rate limiter blocking legitimate users)
- 7.6 Runbook 6 — PgBouncer / Connection Pool Exhaustion (all endpoints slow, 504 or no response)
- 7.7 Runbook 7 — Stripe Webhook 400 "constructEvent FAILED" 100% of callbacks
- 7.8 Runbook 8 — 403 "Cross-origin request blocked" for internal POSTs (CSRF middleware firing false-positive)
- 7.9 Disaster Recovery — Formal RPO / RTO + Catastrophic Outage Protocol
- 7.10 Escalation matrix

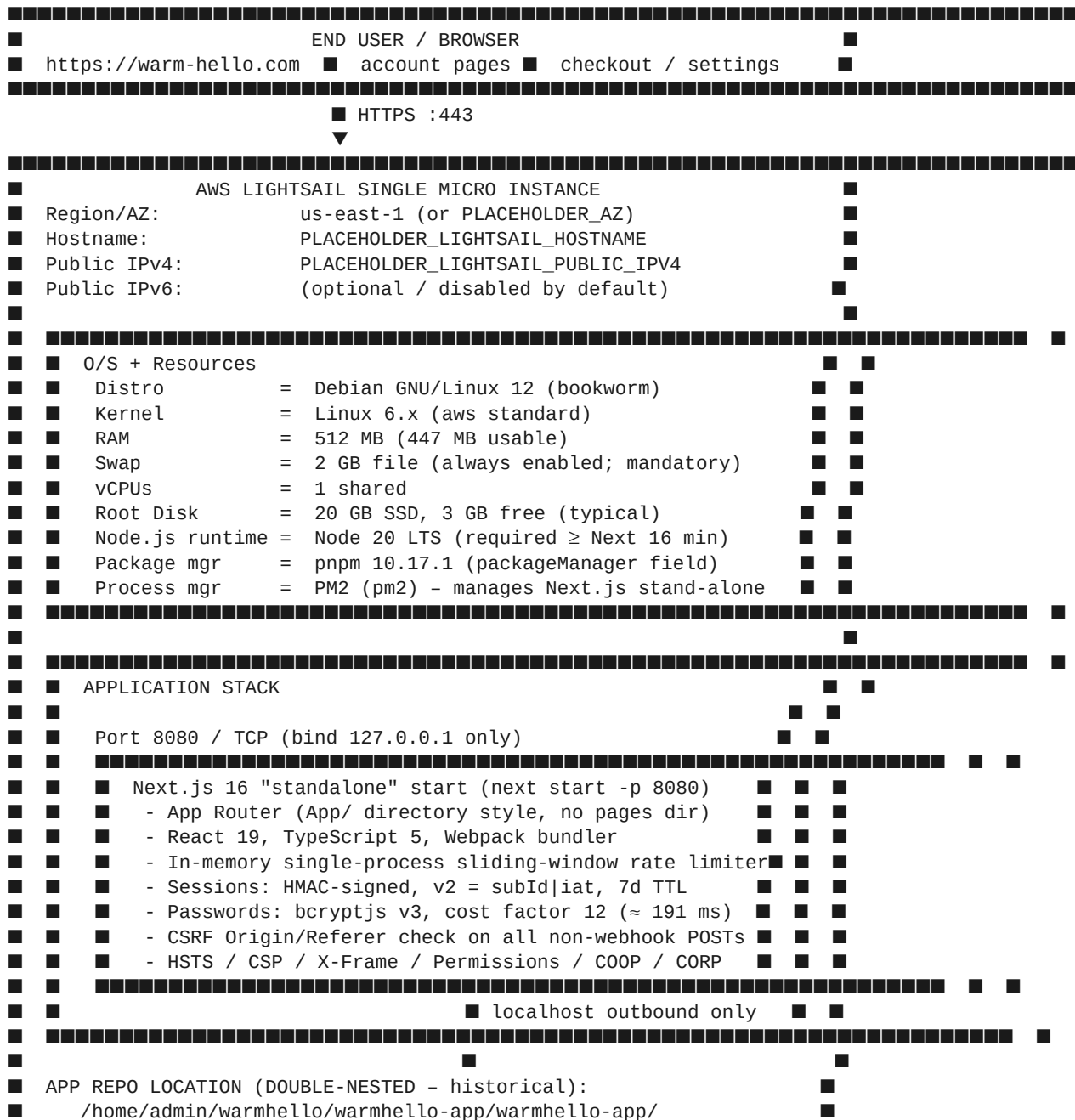
8 Emergency Contacts & Escalation

- 8.1 Internal Escalation Path — Warm-Hello
- 8.2 Third-Party Vendor Support Channels
- 8.3 Legal / Compliance / Breach Notification Contacts
- 8.4 P1 Incident Call Bridge Information
- 8.5 Open a P1 incident checklist
- 8.6 Placeholder Fill-In Table — Once Before Go-Live

Chapter 1 — Architecture & Infrastructure Map

Warm-Hello ([warmhello-app](#)) is a standalone, single-instance Next.js 16 full-stack application for family check-ins. It has no microservices, no kubernetes, no service mesh, no internal queue daemon, and no load balancer today. It is a "deploy a git pull + pm2 restart" topology.

1.1 System Diagram (Text)



[illegible]

No load balancer, no CDN, no Redis, no container runtime (no Docker, no OCI, no containerd), no queue worker (no separate process, no BullMQ, no Sidekiq). All background scheduling is delegated **externally** to Upstash QStash which POSTs back into `/api/jobs/*`. Every inbound request (web page, API, webhook) terminates in the **same single** Next.js Node process under PM2.

— — —

1.2 Hosting & Resources

Item	Value
Cloud provider	AWS Lightsail
Instance plan	\$3.50/month / 512 MB / 1 vCPU / 20 GB SSD (micro)
Region / AZ	PLACEHOLDER_REGION — e.g. us-east-1a . Retrieve with <code>aws lightsail get-instances</code> on CLI or Lightsail Console → Instance → Details tab.
Distro / OS	Debian 12 (bookworm)
Installed prerequisites	Node.js 20+ LTS, pnpm 10.17.1 (corepack enable required if node <22), git, PM2 global, 2 GB swapfile

Item	Value
Swapfile	Mandatory. Location <code>/swapfile</code> 2 GB, <code>swapon</code> in <code>/etc/fstab</code> . Without swap, Next build reliably OOMs with SIGABRT at ~170 MB RSS even with <code>max-old-space-size=2048</code> .
Container / Runtime	No containers. Node.js bare metal on host OS . <code>next start -p 8080</code> is the process.
Reverse proxy	NONE TODAY. App listens on loopback 127.0.0.1:8080. Lightsail default networking exposes the public IP via its own managed frontend / HTTPS wrapper. If custom TLS/Certbot is ever added, install nginx or caddy and <code>proxy_pass 8080</code> .
Disk free threshold	Warn at < 4 GB, critical at < 2 GB. <code>.next/cache/webpack</code> and <code>node_modules</code> dominate space.
CPU burst budget	Very low on micro plan. Sustained >80% CPU over 10 min → Lightsail starts throttling burst credits → latency 3× normal. Threshold documented in §5.3.

1.3 Network Topology

1.3.1 IP allocations

Public IPv4 (Lightsail Elastic IP): `PLACEHOLDER_LIGHTSAIL_PUBLIC_IPV4` (SSH + HTTPS from internet)
 Private IPv4 (VPC / Lightsail VPC): `PLACEHOLDER_LIGHTSAIL_PRIVATE_IPV4` (irrelevant single-node, no peering)
 IPv6: `disabled-by-default` (enable only after CSP tested)

Update placeholders by running on the SSH host: `bash curl -4 ifconfig.co # Public IPv4`
`hostname -I # Private + VPC`

1.3.2 Port requirements

Open firewall **INBOUND** (Lightsail console → Networking → Firewall):

P o r t	P r o t o c o l	Source CIDR	Purpose	Required?
22	TCP	<code>0.0.0.0/0</code> (restrict to admin office IP in Stage 2 hardening)	SSH admin access	Yes, mandatory today
80	TCP	<code>0.0.0.0/0, ::/0</code>	HTTP-01 ACME (Certbot) + redirect to HTTPS	Yes if custom TLS; no if using Lightsail managed HTTPS only
443	TCP	<code>0.0.0.0/0, ::/0</code>	HTTPS user traffic	Yes (user-facing)

P o r t	P r o t o c o l	Source CIDR	Purpose	Required?
8080	TCP	127.0.0.1/32 only, NOT WIDE OPEN	Node.js Next.js stand-alone listener. NEVER expose 8080 to the internet; always loopback only or filtered to reverse-proxy host.	Yes, but do NOT add to Lightsail firewall

OUTBOUND EGRESS (required by the app for integrations, DNS already open in default Lightsail):

P o r t	Pro tocol	Dest	Purpose
53	UDP	AWS/VPC resolver / 8.8.8.8 fallback	DNS (do not block — Next build fetches packages via pnpm using DNS)
443	TCP	*.supabase.com *.pooler.supabase.com	Postgres over TLS via PgBouncer (all DB is TLS)
443	TCP	api.stripe.com js.stripe.com checkout.stripe.com hooks.stripe.com	Stripe billing + hosted checkout links + webhook signing
443	TCP	api.telnyx.com	Telnyx SMS sends outbound check-in links
465	TCP	email-smtp.*.amazonaws.com	SES SMTP TLS send (password auth via IAM creds)
443	TCP	qstash.upstash.io *.qstash.upstash.io	Upstash QStash schedule + publish
443	TCP	registry.npmjs.org get.pnpm.io nodejs.org github.com	Only needed during build / deploy (pinned versions via lockfile)

1.3.3 Firewall / Security group rules (Lightsail default firewall)

Default template rules already work; for hardening additionally add:

1. Restrict SSH :22 CIDR from 0.0.0.0/0 → [/32](https://127.0.0.1/32) admin workstation static IP.
2. Confirm 8080 is NEVER in the firewall list (double-check this; leaks kill CSRF defense).

1.3.4 Domain & DNS

Record	Type	Target / Value	TTL
warm-hello.com	A	PLACEHOLDER_LIGHTSAIL_PUBLIC_IPV4	300 (5 min, low for rollover)

Record	Type	Target / Value	TTL
www.warm-hello.com	CNAME	warm-hello.com.	300
(optional mail)	MX	PLACEHOLDER_MX_PROVIDER e.g. aspmx.l.google.com for Google Workspace if using Google mail	3600
(DKIM/SES)	PLACEHOLDER_DKIM_SELECTOR._domainkey	CNAME → AWS SES DKIM record generated in SES console → Verified identities → DKIM tab	1800
(SPF/TXT)	warm-hello.com TXT	"v=spf1 include:amazonses.com ~all" (combine carefully if using other MX providers, use ~all during cutover then -all after confirmed)	3600
(DMARC/TXT)	_dmarc.warm-hello.com TXT	"v=DMARC1; p=quarantine; sp=quarantine; rua=mailto:postmaster@warm-hello.com; pct=100" → after 30 days green → p=reject	3600

Confirm DNS live from your laptop before first HTTPS cutover:

```
dig +short A warm-hello.com
dig +short CNAME www.warm-hello.com
dig TXT +short warm-hello.com
```

1.4 Horizontal Scaling Caveat (Today = NOT SUPPORTED)

The rate limiter uses an in-memory `Map<string, number[]>` — no Redis. On a single PM2 process this is deterministic and works. If a second Lightsail VM or a second `pm2 start -i max` (cluster mode) is ever added:

1. Session cookies still work (they are HMAC-signed + server-verified, not server-stored).
2. **Rate limiter buckets will not synchronize**, meaning each process will track rates independently → limits become effectively N× looser per load balancer split.
3. Fix: move `lib/rate-limit.ts` from `Map` → Upstash Redis or use Upstash `SET` + `EXPIRE INCR` Lua sliding window.

Until this is addressed, **only run a single PM2 process, single VM instance**. Do not add horizontal replicas before swapping the rate limiter store.

Chapter 2 — Deployment & CI/CD Pipeline

Warm-Hello has **no CI/CD**, **no staging environment**, and **no Terraform or IaC** today. Everything is manual `git pull` → `build` → `pm2 restart` executed over SSH. All environments are flat (single production instance, separate laptops for dev).

This document contains:

- §2.1 Build process (reproducible local build → build artifact → prod build)

- §2.2 Fresh environment provisioning (from zero to reachable)
- §2.3 Production deployment workflow
- §2.4 Staging vs. production paths (both = same blueprint, none in use today)
- §2.5 Rollback strategy (pinpoint sha rollback)
- §2.6 Recommended future CI/CD GitHub Actions starter YAML

2.1 Build Process

2.1.1 Toolchain

Prerequisites required on every build host (dev + prod identical):

```
node    ≥ 20 LTS
pnpm    10.17.1 (enforced by package.json "packageManager": "pnpm@10.17.1")
git     ≥ 2.38
Linux or macOS (Windows WSL2 Ubuntu 22+ also works for local dev)
```

If Node ≥ 20 but no `pnpm` yet:

```
corepack enable      # ships with node 16.18+, makes pnpm/yarn global
corepack prepare pnpm@10.17.1 --activate
```

2.1.2 One-shot local build (dev laptop verification)

```
cd warmhello-app/
cp .env.example .env      # Fill in at least JOB_SIGNING_SECRET ≥32 chars + DATABASE_URL valid
pnpm install --frozen-lockfile
pnpm exec prisma validate      # exit 0 required
pnpm exec prisma generate      # writes @prisma/client under node_modules
pnpm exec tsc --noEmit         # exit 0 required (type check)
NODE_OPTIONS="--max-old-space-size=2048" pnpm build # writes .next/
pnpm lint                     # exit 1 expected due to 7 pre-existing untouched lint errors (documented)
```

Build artifact produced = directory tree: `.next/` (Next stand-alone output, NOT committed to git; regenerated every fresh build).

2.1.3 512 MB micro build constraint (CRITICAL — must read)

On 512 MB VM, Node's V8 default heap auto-sizes to ~380 MB; Next build phase OOMs with exit 134 (SIGABRT) unless we raise `max-old-space-size` to **2048** (V8 allows it, and the 2 GB swapfile absorbs evicted page cold).

Always prefix:

```
NODE_OPTIONS="--max-old-space-size=2048" &lt;command>;
```

Works on `pnpm install` (large install phases), `next build`, `next start`, `prisma generate` (never broke without; harmless to include), `tsc --noEmit` at very large project sizes.

2.2 Environment Provisioning (Fresh Lightsail VM from scratch)

Assumes brand new Debian 12 Lightsail micro instance created via AWS Console. Connect SSH as `admin@PUBLIC_IPV4`.

Step 1/7 — Update OS + add 2 GB swapfile (mandatory on micro)

```
set -euxo pipefail
sudo apt-get update -y && sudo DEBIAN_FRONTEND=noninteractive apt-get -y dist-upgrade
sudo fallocate -l 2G /swapfile && sudo chmod 0600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo "/swapfile swap swap defaults 0 0" | sudo tee -a /etc/fstab
free -h ; swapon --show      # Verify Swap: total=2.00GiB
sudo apt-get install -y curl git build-essential ufw ca-certificates gnupg
```

Step 2/7 — Install Node.js 20 LTS + pnpm 10.17.1 via nodesource

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
node -v      # expect v20.x.x
corepack enable
corepack prepare pnpm@10.17.1 --activate
pnpm -v      # expect 10.17.1
```

Step 3/7 — Install PM2 global and make it auto-start on reboot

```
sudo npm install -g pm2
pm2 --version
pm2 startup systemd -u admin --hp /home/admin
# paste the line it prints back (starts with "sudo env PATH=...")
```

Step 4/7 — UFW firewall (extra defense even behind Lightsail firewall)

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw limit 22/tcp comment 'SSH (rate limited 6/30s)'
sudo ufw allow 80/tcp comment 'HTTP for Let us Encrypt / redirect'
sudo ufw allow 443/tcp comment 'HTTPS user traffic'
# DO NOT ufw allow 8080/tcp
echo "y" | sudo ufw enable
sudo ufw status numbered verbose
```

Step 5/7 — Clone repo, install once

```
mkdir -p /home/admin/warmhello/warmhello-app/
cd /home/admin/warmhello/warmhello-app/
git clone https://github.com/MarioAltamira/warmhello-app.git
# Result: /home/admin/warmhello/warmhello-app/warmhello-app/ (double-nested historical path)
cd /home/admin/warmhello/warmhello-app/warmhello-app/
NODE_OPTIONS="--max-old-space-size=2048" pnpm install --frozen-lockfile
```

Step 6/7 — Place .env with all production credentials

Copy from [03-configuration-and-environment-variables.md](#) §3.3. Verify file perms:

```
chmod 0600 .env
ls -la .env
# -rw----- 1 admin admin ... .env
```

Step 7/7 — Bootstrap DB + first start

```
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec prisma migrate deploy
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec prisma generate
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec next build
```

```
pm2 start "NODE_OPTIONS='--max-old-space-size=2048' pnpm exec next start -p 8080" --name warmhello --time
pm2 save
sleep 25
curl -sS http://127.0.0.1:8080/api/health
# Expect 200 with {"ok":true,"integrations":{"prisma":true,"stripe":true,"sms":true,"email":true,"qstash":true}}
---
```

2.3 Standard Production Deployment Workflow (git push → SSH deploy)

Used after every new commit on [main](#). Run inside an already-open admin SSH session on the Lightsail VM.

```
set -euo pipefail
cd /home/admin/warmhello/warmhello-app/warmhello-app/

# === 1/5 Snapshot previous state for easy rollback ===
PREV_SHA=$(git rev-parse HEAD)
echo "PREVIOUS_ROLLBACK_SHA=$PREV_SHA"
echo "Write this sha down – you can 'git reset --hard $PREV_SHA' if rollback needed."

# === 2/5 Fetch latest, reset working tree clean (discard any runtime local edits) ===
git fetch --all --prune
git reset --hard origin/main
git clean -fdx # CAREFUL: removes any uncommitted files EXCEPT untracked .env (check .gitignore first; .env

# === 3/5 Install + Prisma ===
NODE_OPTIONS="--max-old-space-size=2048" pnpm install --frozen-lockfile
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec prisma generate
# Run migrations ONLY IF prisma/migrations/ folder has new entries since last deploy:
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec prisma migrate deploy
# (exit 0 with "0 pending" if no new migrations – safe to always run)

# === 4/5 BUILD + SWAP ===
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec next build 2>&1 | tail -n 40
# Build success prints "(Static | Dynamic | λ Server)" grid. No SIGABRT.
pm2 delete warmhello || true
pm2 start "NODE_OPTIONS='--max-old-space-size=2048' pnpm exec next start -p 8080" --name warmhello --time
pm2 save

# === 5/5 HEALTH CHECK (sleep 25s mandatory) ===
echo "sleeping 25 seconds for Next.js cold start"
sleep 25
for i in 1 2 3 4 5; do
  echo "--- attempt $i at $(date '+%H:%M:%S') ---"
  curl -sS http://127.0.0.1:8080/api/health || echo "(curl wait – Next still compiling routes)"
  echo ""
  sleep 5
done
echo "=== tail pm2 logs 100 lines ==="
pm2 logs warmhello --lines 100 --nostream --raw | tail -n 120
```

Timing: total deploy 5m 30s → 9m typical on 512 MB micro (build is ~3.5 → 6m; npm install dominates if many new deps; cold start after pm2 start = ~10 → 14s; hence mandatory sleep 25 before first curl to avoid false curl7).

2.4 Staging vs. Production (paths today + recommendation)

Today: no staging

- Single branch = `main` is production. There is **no GitHub Actions, no preview deployments, no staging VPS, no staging DB**.
- Staging environment = developer laptop `pnpm dev` against `DATABASE_URL` dev-only Supabase branch. Dev DB branch should be created manually via Supabase Console → Database → Branches → "Create branch from main".

Recommendation (medium-term, not blocking anything today)

Add a 2nd small Lightsail instance `staging-warmhello` with its own:

1. Separate `.env` → points `DATABASE_URL` at a Supabase staging branch DB clone, uses Stripe **test mode** SK keys, Telnix **test messaging profile**, QStash `staging-` namespace token.
2. Separate PM2 process name `warmhello-staging` on port **8081** bind 127.0.0.1.
3. Deploy pipeline identical §2.3 but tracks `staging` branch instead of `main`.
4. Cutover promotion flow: `staging deploy` → `verify on URL` → `PR merge main` → `prod deploy`.

2.5 Rollback Strategy

Two rollback speeds depending on failure surface.

A — Fast / Preferred: Roll back source code + rebuild (≤ 10 min)

When the NEW build boots but misbehaves (e.g., 500s on homepage, DB schema mismatch we can revert by applying schema backwards through prisma, new rate limiter too aggressive, etc.):

```
set -euo pipefail
cd /home/admin/warmhello/warmhello-app/warmhello-app/
# 1. Record the broken sha, snap previous good sha
BROKEN=$(git rev-parse HEAD)
echo "Broken deploy at $BROKEN"
# 2. Target = last known good. Pick one:
TARGET=PREV_SHA_YOU_WROTE_DOWN_IN_STEP_1      # (Recommended. You wrote it down!)
# - or -
TARGET=HEAD~1                                  # last commit (if you know 1 commit back is ok)
# - or -
TARGET=ea93ad2                                  # named good sha from commit log

# 3. Hard reset and rebuild
git reset --hard $TARGET
git clean -fdx
NODE_OPTIONS="--max-old-space-size=2048" pnpm install --frozen-lockfile
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec prisma generate
# Note: Prisma migrations DOWN requires manual resolve IF a broken migration was applied.
# Run prisma migrate status FIRST to see if down is needed, else skip.
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec prisma migrate status
# If status says "1 migration applied but not in schema yet" → manual §4.2 procedure
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec next build
pm2 delete warmhello || true
pm2 start "NODE_OPTIONS='--max-old-space-size=2048' pnpm exec next start -p 8080" --name warmhello --time
pm2 save
sleep 25 &&& curl -sS http://127.0.0.1:8080/api/health
```

B — Instant / Emergency: Revert PM2 to previous build folder (≤ 1 min, only preserves 1 prior build)

If you had foresight to keep the previous `.next/` before overwriting with a new build:

```
# Run BEFORE new build, run this once per deploy as insurance:
cp -a .next .next.predeploy
# Then if new build fails:
rm -rf .next && mv .next.predeploy .next
pm2 restart warmhello --time
sleep 25 && curl -sS http://127.0.0.1:8080/api/health
```

Warning: This trick only works if no new migrations were applied between the two builds. If DB schema changed, the `.next.predeploy` code will emit Prisma "column X does not exist" errors and you must immediately move to Rollback A + manual Prisma migrate resolve down.

C — Prisma-specific migration rollback (last resort, ≥ 30 min)

See §4.2 Database Migration Rollback procedure.

2.6 Future Recommended CI/CD — GitHub Actions Starter

Not configured today. Drop the following into `.github/workflows/ci.yml` to get a **main-branch gate that blocks on type-check failure + invalid Prisma schema**. Do NOT run `next build` on 2-core GitHub runners against 512 MB micro memory quota (will pass but slow and costly). Add a deploy step via SSH deploy key only **after** staging env exists.

```
name: Warm-Hello CI
on:
  push:
    branches: [main, staging]
  pull_request:
jobs:
  typecheck-and-lint:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
        with:
          version: 10.17.1
          run_install: false
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: pnpm
      - name: Install
        run: pnpm install --frozen-lockfile
        env:
          NODE_OPTIONS: "--max-old-space-size=4096"
      - name: Fake minimal env (only vars needed for zod schema parse pass)
        run: |
          echo "JOB_SIGNING_SECRET=ci-only-dev-secret-min-32-characters-0123456789" >> .env
      - name: Prisma validate
        run: pnpm exec prisma validate
      - name: Prisma generate
        run: pnpm exec prisma generate
```

- name: tsc --noEmit (hard failure on any TS error)
run: pnpm exec tsc --noEmit
- name: Lint (informational, non-blocking due to 7 pre-existing untouched errors)
run: pnpm lint || true

Chapter 3 — Configuration & Environment Variables

All runtime configuration is parsed with Zod at import-time by `lib/env.ts`. If any required key is invalid, process will **throw at first** `import env` (usually PM2 restart loop; you will see errors like `Invalid environment variables → JOB_SIGNING_SECRET: Expected string, length ≥ 32 in warmhello-error-0.log`).

3.1 Secrets Management — Today

Current storage location: plaintext file `.env` `chmod 0600`

- Path on server: `/home/admin/warmhello/warmhello-app/warmhello-app/.env`
- Path on dev laptops: repo root `warmhello-app/.env`, already `.gitignore`d
- Owner on server: `admin:admin`, mode `-rw-----` (only owner read/write)
- NO secrets manager, NO Vault, NO AWS Secrets Manager today.
- `.env` IS explicitly listed in `.gitignore`. NEVER commit this file.

3.1.1 Recommended hardening (6-12 month)

Move only the *production 11 keys below* into AWS Secrets Manager → load at boot via a small `envsubst` step in PM2 script. Keep dev `.env` on laptops as today (local-only entropy). This is a **future recommendation only** — not required for current operating handoff.

The 11 actual production-only secret values that MUST migrate first when you adopt a secrets manager:

1. `DATABASE_URL` (Supabase pooled + password)
2. `DIRECT_URL` (Supabase non-pooled + password)
3. `JOB_SIGNING_SECRET`
4. `STRIPE_SECRET_KEY`
5. `STRIPE_WEBHOOK_SECRET`
6. `TELNYX_API_KEY`
7. `TELNYX_WEBHOOK_SECRET`
8. `SMTP_USERNAME` (AWS SES IAM access key id)
9. `SMTP_PASSWORD` (AWS SES IAM smtp password)
10. `QSTASH_TOKEN`

11. .env file integrity — do not append # comments after value=tokens

3.2 Environment Matrix (Complete Zod Key List)

Source of truth = [env.ts](file:///C:/Users/mario/Documents/trae_projects/WarmHello/warmhello-app/lib/env.ts).
 Template =

[.env.example](file:///C:/Users/mario/Documents/trae_projects/WarmHello/warmhello-app/.env.example).

Variable	Required	Type	Default value if unset	Dev (local laptop)	Staging (future)	Production
NODE_ENV	soft	development test production	development	development	production	production
APP_URL	soft	full URL incl scheme	http://localhost:8080	http://localhost:8080	https://staging.warm-hello.com	https://warm-hello.com
NEXT_PUBLIC_APP_URL	soft	full URL incl scheme (SENT TO BROWSER JS BUNDLE)	http://localhost:8080	http://localhost:8080	https://staging.warm-hello.com	https://warm-hello.com
DATABASE_URL	conditional	Postgres URI, pooled, starts postgres://	undefined (blank) → DB-disabled mode for pure front-end dev	Supabase dev branch pooled URI	Supabase staging branch pooled URI	Supabase main pooled URI (*.pooler.supabase.com)
DIRECT_URL	conditional	Postgres URI, non-pooled direct connect, starts postgres://	undefined (blank)	Supabase dev branch direct URI	Supabase staging branch direct URI	Supabase main direct URI
JOB_SIGNING_SECRET	■■■ ABSOLUTELY REQUIRED	min 32 characters, high-entropy random (NO DEFAULT)	Zod throws	min 32 chars, unique dev-only	min 32 chars, unique staging-only	min 32 chars, unique production only. IF PROD ENV LEN <32 THE APP WILL NOT BOOT.

Variable	Required	Type	Default value if unset	Dev (local laptop)	Staging (future)	Production
STRIPE_SECRET_KEY	conditional	starts <code>sk_live_</code> or <code>sk_test_</code>	<code>undefined</code> → billing disabled	<code>sk_test_*</code> from Stripe test dashboard	<code>sk_test_*</code>	<code>sk_live_*</code>
STRIPE_WEBHOOK_SECRET	conditional	starts <code>whsec_</code>	<code>undefined</code> → webhook verify always fails (good, do not set unless you have an endpoint configured in Stripe)	test webhook <code>whsec_*</code>	test webhook <code>whsec_*</code>	live webhook <code>whsec_*</code>
STRIPE_PRICE_ID	conditional	single legacy default price id <code>price_*</code>	<code>undefined</code>	test mode default price	test mode default price	live default monthly (can be empty if USD/CAD specific vars below populated instead)
STRIPE_PRICE_ID_USD_MONTHLY	conditional	<code>price_*</code> USD monthly	<code>undefined</code>	test price	test price	live price
STRIPE_PRICE_ID_USD_ANNUAL	conditional	<code>price_*</code> USD annual	<code>undefined</code>	test price	test price	live price
STRIPE_PRICE_ID_CAD_MONTHLY	conditional	<code>price_*</code> CAD monthly	<code>undefined</code>	test price	test price	live price
STRIPE_PRICE_ID_CAD_ANNUAL	conditional	<code>price_*</code> CAD annual	<code>undefined</code>	test price	test price	live price

Variable	Required	Type	Default value if unset	Dev (local laptop)	Staging (future)	Production
TELYX_API_KEY	conditional	starts KEY0*... or KEY1*... per Telnix v2	<code>undefined</code> → SMS disabled	test profile API key	test profile API key	live messaging profile API key
TELYX_FROM_NUMBER	conditional	E.164 format +1NNNNNNNNNN	<code>undefined</code> → SMS disabled	Telnix test number	Telnix staging number	Telnix production 10DLC campaign number
TELYX_WEBHOOK_SECRET	conditional	string returned when creating webhook in Telnix portal	<code>undefined</code> → Telnix webhooks not verified (reject all)	test webhook secret	test webhook secret	live webhook secret
SHORTLINK_BASE_URL	conditional	URL scheme+host	<code>undefined</code>	<code>http://localhost:8080</code>	<code>https://staging.warm-hello.com</code>	<code>https://warm-hello.com</code>
EMAIL_FROM_ADDRESS	soft	RFC-5321 email address, address used as <code>From:</code>	<code>sales@warm-hello.com</code>	<code>sales-dev@warm-hello.com</code>	<code>sales-staging@warm-hello.com</code>	<code>sales@warm-hello.com</code>
SMTP_HOST	conditional	hostname	<code>undefined</code> → email globally disabled (no signup welcome, no password reset, no trial emails)	SES sandbox smtp hostname	SES smtp hostname	<code>email-smtp.us-east-1.amazonaws.com</code> or SES-sandbox-removed matching region
SMTP_PORT	conditional	integer positive, must match host	<code>undefined</code>	465	465	465 (TLS mandatory wrapper, NO STARTTLS 587)
SMTP_USERNAME	conditional	string SES gives you	<code>undefined</code> → email disabled	SES sandbox IAM SMTP user key id	SES staging IAM SMTP user key id	SES production IAM SMTP user key id (AKIA...)
SMTP_PASSWORD	conditional	string	<code>undefined</code> → email disabled	SES sandbox SMTP password	SES staging SMTP password	SES production SMTP password (NOT console login; SMTP credential password only)

Variable	Required	Type	Default value if unset	Dev (local laptop)	Staging (future)	Production
QSTASH_URL	soft	URL, scheme+host	https://qstash.upstash.io (always leave this default)	default	default	default
QSTASH_TOKEN	conditional	starts Bearer... → the Upstash token part (paste WITHOUT "Bearer" prefix; token is Bearer <X> header built internally)	undefined → jobs disabled, timers silent (qstash status in health endpoint will be false/red)	dev QStash token	staging QStash token	production QStash token

3.2.1 Derived integration status flags

[/api/health](#) computes per-integration booleans from these env vars. True iff the marked sub-vars are all non-empty.

Health key "integrations.X"	True if
prisma	DATABASE_URL set
stripe	STRIPE_SECRET_KEY set AND STRIPE_PRICE_ID (or 1+ of 4 USD/CAD MONTHLY/ANNUAL price ids) set
sms	TELNYX_API_KEY set AND TELNYX_FROM_NUMBER set
email	EMAIL_FROM_ADDRESS set AND SMTP_HOST AND SMTP_PORT AND SMTP_USERNAME AND SMTP_PASSWORD all set
qstash	QSTASH_TOKEN set

All 5 integrations=true in production. The app runs in "degraded" dev/demo mode with a subset false during front-end-only work.

3.3 Production .env file — minimum required block

Copy exactly this pattern (**UNQUOTED values only** — zod parser strips stray wrapping quotes but keep paste-clean). Paste values without a trailing slash on URLs.

```
# ===== Node runtime =====
NODE_ENV=production

# ===== URLs =====
APP_URL=https://warm-hello.com
NEXT_PUBLIC_APP_URL=https://warm-hello.com
SHORT_LINK_BASE_URL=https://warm-hello.com

# ===== Database (Supabase pooled + direct) =====
DATABASE_URL=postgres://postgres.mjcgmecafwhlzqglcmqj:PLACEHOLDER_SUPABASE_POOLER_PASSWORD@aws-0-ca-central-1-...
DIRECT_URL=postgres://postgres.mjcgmecafwhlzqglcmqj:PLACEHOLDER_SUPABASE_DIRECT_PASSWORD@aws-0-ca-central-1-...

# ===== Jobs / CSRF / Unsubscribe signing =====
# 1-time generation (run from laptop once): node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

```
# MUST BE >= 32 chars or app refuses to boot.
JOB_SIGNING_SECRET=PLACEHOLDER_JOB_SIGNING_SECRET_MIN_32_CHARS_GENERATED_VIA_NODE_ABOVE

# ===== Stripe LIVE =====
STRIPE_SECRET_KEY=sk_live_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
STRIPE_WEBHOOK_SECRET=whsec_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
STRIPE_PRICE_ID=price_XXXXXXXXXXXXXXXXXXXXXXXXXXXX (leave blank if you use only the 4 specific ones)
STRIPE_PRICE_ID_USD_MONTHLY=price_XXX
STRIPE_PRICE_ID_USD_ANNUAL=price_XXX
STRIPE_PRICE_ID_CAD_MONTHLY=price_XXX
STRIPE_PRICE_ID_CAD_ANNUAL=price_XXX

# ===== Telnyx LIVE 10DLC SMS =====
TELNYX_API_KEY=KEY00000000000000000000000000000000
TELNYX_FROM_NUMBER=+15555551234
TELNYX_WEBHOOK_SECRET=PLACEHOLDER_TELNYX_WEBHOOK_SECRET

# ===== Email (AWS SES SMTP) =====
EMAIL_FROM_ADDRESS=sales@warm-hello.com
SMTP_HOST=email-smtp.us-east-1.amazonaws.com
SMTP_PORT=465
SMTP_USERNAME=AKIAIOSFODNN7EXAMPLE
SMTP_PASSWORD=XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

# ===== Upstash QStash =====
QSTASH_URL=https://qstash.upstash.io
QSTASH_TOKEN=PLACEHOLDER_UPSTASH_QSTASH_TOKEN_PROD
```

After pasting, run a parse validation:

```
set -a ; . ./env ; set +a
node -e "console.log('len JOB_SIGNING_SECRET=', '${JOB_SIGNING_SECRET}'.length)"
# Expect 32+ (64 if you used crypto.randomBytes(48).toString('base64url'))
```

...

3.4 Local Dev .env (Developer Laptop)

Minimal passable parse:

```
NODE_ENV=development
APP_URL=http://localhost:8080
NEXT_PUBLIC_APP_URL=http://localhost:8080
DATABASE_URL=postgres://postgres:postgres-dev-db-password@localhost:5432/warmhello_dev
# or use Supabase dev branch pooled URL
DIRECT_URL=
JOB_SIGNING_SECRET=local-dev-only-min32-changeme-dontuseinprod123
```

All other keys blank = integrations disable themselves gracefully via health false. Front-end still renders; you can test pages without SMS/Stripe.

...

3.5 Changing an environment variable — live rollout procedure

Do NOT hot-edit `.env` and expect PM2 to pick it up automatically. It won't.

1. Edit `.env`
2. Confirm `chmod 0600 .env`

3. Rebuild + restart process (must rerun next build because some `NEXT_PUBLIC_*` vars are baked into the browser JS bundle at build time):

```
cd /home/admin/warmhello/warmhello-app/warmhello-app/
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec next build
pm2 restart warmhello --time
sleep 25 ; curl -sS http://127.0.0.1:8080/api/health
```

Exceptions (keys NOT baked into build, so you can get away with `pm2 restart warmhello` skipping build if the only change was purely server-side keys like `DATABASE_URL` or `SMTP_PASSWORD`):

- Purely server-env keys (not prefixed `NEXT_PUBLIC_`, and not referenced inside `middleware.ts` or route handlers that hit client bundle code) — still rebuild is **safer* and only 4 minutes; always rebuild for anything non-trivial.

Chapter 4 — Database & Data Operations

Warm-Hello uses a **single PostgreSQL 16** database hosted by Supabase in AWS ca-central-1. Supabase manages: automated backups, PITR, OS updates, storage, replication. The Warm-Hello app uses two Postgres endpoints (same logical DB):

1. **Pooled** — `DATABASE_URL *.pooler.supabase.com :5432` — PgBouncer transaction-mode pooling. Used by **ALL runtime Prisma queries in the app** (the 99% path). Enforces max 15 connections in the URI `?connection_limit=15` on 512 MB micro.
2. **Direct** — `DIRECT_URL *.supabase.com :5432` — 1:1 raw connection. Used by `prisma migrate deploy` DDL, `prisma db push`, and `prisma migrate resolve`. Never used for user queries.

4.1 Engine Specs

Spec	Value
Database engine	PostgreSQL 16.x (Supabase always tracks latest stable minor)
DBaaS provider	Supabase (runs on AWS ca-central-1)
Logical DB name	<code>postgres</code> (default supabase DB name)
Postgres user	<code>postgres.mjcgmecafwhlzqglcmqj</code> (per Supabase pattern: project owner auto-generated role)
Pooled Hostname	<code>aws-0-ca-central-1.pooler.supabase.com</code>
Pooled Port	5432, PgBouncer mode transaction (NOT session). Prepared statements OK per Supabase docs because they wrap server-side.
Direct Hostname	<code>aws-0-ca-central-1.supabase.com</code>
SSL	ALWAYS on. Prisma auto-configures <code>sslmode=require</code> for Supabase URLs. No plaintext anywhere.
ORM	Prisma 6.19.x. Schema path: <code>prisma/schema.prisma</code>

Spec	Value
Prisma client location	auto-generated into <code>node_modules/@prisma/client</code> each <code>prisma generate</code> run
Connection pool size (app-level Prisma)	<code>connection_limit=15</code> on pooled URL (default would be <code>num_cpus*2+1</code> which is 3; bump to 15 for 3x burst headroom on PgBouncer)
Timeouts	Prisma default <code>statement_timeout</code> 30 s (global). Prisma <code>pool_timeout</code> 10 s.
Row-level access control	NONE in Postgres RLS today (all access trusted via app server ORM only). The DB is not exposed to browser Supabase SDK client; no anon/key URL in JS bundle.
Supabase Auth	DISABLED. We do not use Supabase Auth JS SDK, magic links, JWT, or RLS. Auth is handled app-server-side bcryptjs + HMAC-signed cookies only.

4.2 Migration Procedures

Prisma migration files live in `prisma/migrations/YYYYMMDDHHmmSS_migration_name/migration.sql`. They are applied server-side with `prisma migrate deploy`, NEVER `prisma migrate dev` (dev-only command for creating migrations, not applying).

4.2.1 Apply pending migrations (deploy-time only)

Run **BEFORE** `next build` on deploy step §2.3. It is safe to always run; `0 pending migrations` exits 0.

```
cd /home/admin/warmhello/warmhello-app/warmhello-app/
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec prisma migrate deploy --schema prisma/schema.prisma
```

4.2.2 Audit migrations applied on target DB

```
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec prisma migrate status
# Expected output example:
# 8 migrations found in prisma/migrations
#
# Following migration have been applied:
# 20260820170000_init
# 20260902_add_password_hash
# 20260902_add_password_auth_enum_values
#
# No pending migrations to apply.
```

4.2.3 Creating a new migration (dev-laptop-only, NEVER on prod)

```
# 1. Edit prisma/schema.prisma on laptop
# 2. Run:
pnpm exec prisma migrate dev --name descriptive_lowercase_name_here
# 3. Creates prisma/migrations/YYYYMMDDHHmmSS_descriptive_lowercase_name_here/migration.sql + updates _prisma
# 4. Review generated SQL before committing; manual edits inside .sql file allowed before commit (for indexes)
# 5. Git commit migration files + schema.prisma together (atomic commit)
```

Never manually edit a migration SQL file once it's been committed to main. That breaks the deterministic Prisma migration checksum; deploy will throw "Checksum mismatch for migration ..." → procedure §4.2.5.

4.2.4 Roll back a deployed migration — last resort (Rare)

Prisma has no built-in "prisma migrate rollback". Steps:

1. Open the migration.sql in `prisma/migrations/.../migration.sql` you want to undo.

2. Write the **inverse** SQL manually: - `ALTER TABLE ... ADD COLUMN foo` → `ALTER TABLE ... DROP COLUMN foo` - `CREATE TABLE bar` → `DROP TABLE bar` - `CREATE UNIQUE INDEX ...` → `DROP INDEX ...`
3. Save as `prisma/migrations/YYYYMMDDHHmmSS_rollback_<name>/migration.sql`.
4. Commit, deploy normally via §4.2.1.
5. Note: data-destroying rollbacks (`DROP TABLE`, `DROP COLUMN`) lose data. For data-preserving rollbacks of DDLs, first inspect column fill % with:

```
SELECT schemaname, tablename, attname, n_distinct, correlation FROM pg_stats WHERE tablename IN ('subscribers
```

4.2.5 Resolve "Checksum mismatch" / "Failed migration"

If deploy broke mid-`prisma migrate deploy` (SSH dropped, VM killed mid-transaction):

```
# 1. Figure out which migration folder is the bad one
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec prisma migrate status

# 2. Option A: If the migration is actually applied (SQL worked) but Prisma thinks failed → mark applied
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec prisma migrate resolve --applied 20260902123456_the_name

# 3. Option B: If the migration truly failed partway (SQL partially applied) → mark rolled-back
# (then manually fix the partial SQL state against production DB with psql)
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec prisma migrate resolve --rolled-back 20260902123456_the_name
```

Always prefer Option A if the actual SQL statements all ran (most common case when killed mid-transaction commit).

4.2.6 Schema validate before commit

```
pnpm exec prisma validate # exit 0 required; catches broken @relation, @id, enum syntax
```

```
---
```

4.3 Backup & Restore

Backups today are **100% Supabase-managed**. You need a Supabase owner/editor login to perform restores.

4.3.1 Automated Backup Schedule (Supabase)

Tier	PITR (point-in-time)	Snapshot backup cadence	Retention
Supabase Free	■ No	Daily automated	2 days
Supabase Pro (\$25/mo)	■ 7-day PITR granular to 1 second	Daily automated	7 days
Supabase Team (\$100/mo)	■ 30-day PITR	Daily automated + on-demand	30 days

Recommendation: Pro tier minimum for production (7 day PITR mandatory). Currently using whatever tier the project is on — confirm this via Supabase console → Project → Billing → Plan.

4.3.2 Ad-hoc manual backups (recommended before every risky migration)

Create a manual `pg_dump` before you run ANY production migrate deploy that has `ALTER` / `ADD COLUMN` with `NOT NULL` / `DROP` / `NEW TABLE CASCADE`.

```
# Run FROM A LAPTOP OR JUMP BOX, NEVER SSH INTO PROD TO DUMP (eats root disk)
DIRECT_URL="postgres://postgres:mjcgmecafwhlzqglcmqj:PLACEHOLDER_SUPABASE_DIRECT_PASSWORD@aws-0-ca-central-1.1.
```

```

DATESTAMP=$(date +%Y%m%d-%H%M%S)
pg_dump "$DIRECT_URL" \
  --format=custom \
  --compress=zstd:9 \
  --no-owner \
  --no-privileges \
  --clean \
  --if-exists \
  -f "warmhello-prod-backup-${DATESTAMP}.dump"
# Copy offline to encrypted s3 bucket or external cold storage
aws s3 cp "warmhello-prod-backup-${DATESTAMP}.dump" s3://PLACEHOLDER_ENCRYPTED_BACKUPS_BUCKET/warmhello/prod/

```

4.3.3 Restore from Snapshot (Supabase console — catastrophic failure)

1. Login to Supabase Console → Project → Settings → Backups
2. Pick restore target (PITR if Pro, else latest daily snapshot)
3. **Warning:** restore is **DESTRUCTIVE TO CURRENT LIVE DB**. Write down the DB size on disk (Settings → Infrastructure → Storage) first.
4. Click Restore.
5. Wait for Supabase (10-40 min depending on DB size; Supabase provisions new instance under-the-hood then swaps).
6. After restore: - Re-validate app's DATABASE_URL pooled (hostname/password do NOT change on snapshot restore; it's same logical project cluster) - `pm2 restart warmhello --time`; sleep 25 ; curl health

4.3.4 Restore from pg_dump dump file (object-level selective restore)

```

DIRECT_URL="postgres://user:pass@host:5432/postgres"
DUMPFIL="warmhello-prod-backup-YYYYMMDD-HHMMSS.dump"
# Inspect first (list objects):
pg_restore --list "$DUMPFIL" | head -n 40
# Restore into a fresh EMPTY clone DB (NEVER OVERWRITE PROD DB WITH RESTORE IN PLACE):
createdb warmhello_restore_test
PGPASSWORD=PLACEHOLDER_RESTORE_DB_PASS pg_restore --no-owner --no-privileges --dbname=warmhello_restore_test

```

4.3.5 Recovery Point / Time Targets

See §7.5 Disaster Recovery for formal RPO / RTO statements.

4.4 Periodic DB Ops Manual Checklist

Every 30 days (repeat monthly cron reminder — set personal task):

1. `psql $DIRECT_URL -c "SELECT pg_size_pretty(pg_database_size(current_database()));"` — record DB size trend (warn if >50% Supabase plan quota).
2. `psql $DIRECT_URL -c "SELECT schemaname, relname, n_live_tup, n_dead_tup, last_autovacuum FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 10;"` — dead tuples blow up planner estimates; if any table `n_dead_tup > 1,000,000` → `VACUUM ANALYZE <table>;`.
3. `psql $DIRECT_URL -c "SELECT pid, username, state, query_start, wait_event_type || ':' || wait_event, query FROM pg_stat_activity WHERE state <> 'idle' ORDER BY query_start;"`

— catch runaway queries (stuck transactions cause PgBouncer pool exhaustion → 503s on app endpoints).

4. Confirm Supabase automated backup is green via Console → Backups → Last success date.
5. Take a manual pg_dump baseline per §4.3.2 and ship to offline encrypted storage.

Chapter 5 — Monitoring, Logging & Observability

No APM vendor (Datadog / New Relic / Sentry) today. All observability is SSH-based using PM2 logs + Linux built-ins plus one public HTTP health check endpoint for uptime monitoring SaaS pings.

5.1 Health Checks

5.1.1 Primary API health endpoint

GET /api/health

- **Port on host:** <http://127.0.0.1:8080/api/health>
- **Public URL (for uptime SaaS):** <https://warm-hello.com/api/health>
- **Response format (JSON):**

```
{
  "ok": true,
  "integrations": {
    "database": true,
    "stripe": true,
    "sms": true,
    "email": true,
    "qstash": true
  },
  "timestamp": "2026-09-02T14:22:08.192Z"
}
```

- **HTTP status = 200 on ALL responses** (ok is inside JSON, not HTTP code — so some log lines still print if 401/403 middleware rejects during maintenance; endpoint itself always returns 200 JSON).
- **What "ok: true" does NOT verify:** this endpoint does NOT attempt a live TCP

connect round-trip against DB, Stripe, SES or QStash. It only checks env vars are set (boolean) — see [health route](file:///C:/Users/mario/Documents/trae_projects/WarmHello/warmhello-app/app/api/health/route.ts) for implementation.

Use for:

1. Lightsail / load balancer simple "is the Node process listening" check
2. Uptime SaaS 60-second interval pings
3. First verification step right after deploy

5.1.2 Deep health checks (run manually by SSH admin when investigation)

Database alive:

```
cd /home/admin/warmhello/warmhello-app/warmhello-app/
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec prisma db execute --stdin <<'EOF'
SELECT 'OK pr_db_1' as check, now() as server_clock, current_database();
EOF
```

Stripe secret key valid:

```
curl -sS https://api.stripe.com/v1/balance -u "$(grep -E '^STRIPE_SECRET_KEY=' .env | cut -d= -f2-):" | head
```

SES SMTP creds valid (will send one test email to admin):

```
node - <<'EOF'
const net = require("net");
const env = Object.fromEntries(require("fs").readFileSync(".env", "utf8").split(/\n/).filter(l=>l && l.trim().length > 0));
console.log("Attempting TLS SMTP connect to", env.SMTP_HOST, env.SMTP_PORT);
// Full EHLO / STARTTLS handshake NOT done here (complex); use openssl s_client instead:
EOF
# Simpler:
openssl s_client -quiet -crlf -connect email-smtp.us-east-1.amazonaws.com:465 </dev/null 2>&1 | head -n 1
# Should print 220 Amazon ESMTP greeting. Exit the manual auth flow by Ctrl-C after confirming handshakes.
```

QStash token valid:

```
TOKEN="$(grep -E '^QSTASH_TOKEN=' .env | cut -d= -f2-)"
curl -sS https://qstash.upstash.io/v2/topics -H "Authorization: Bearer $TOKEN" | head -c 400
```

5.2 Log Aggregation & Rotation

5.2.1 Log locations today (PM2 default)

```
/home/admin/.pm2/logs/
■■■ warmhello-error-0.log      # uncaught exceptions, console.error, zod parse fails, prisma errors, 500s - PM2
■■■ warmhello-out-0.log       # console.log, console.info, request summary lines, 200 OK verbose
```

PM2 is started with `--time` flag so each line is prefixed with UTC timestamp. Example line prefix format:
`2026-09-02T14:31:43: PM2 log: App [warmhello:0] exited with code [0] via signal [SIGINT].`

5.2.2 Live tailing

```
# Show last 200 lines of both logs then follow live:
pm2 logs warmhello --lines 200
# Errors-only view (Unix grep filter):
tail -n 500 -F /home/admin/.pm2/logs/warmhello-error-0.log | grep -v '^$'
```

5.2.3 Rotation policy (MUST DO — NOT ENABLED BY DEFAULT)

Without log rotation `/home/admin/` partition fills with logs, 512MB VM disk is only 20 GB. Enable PM2 logrotate module on EVERY VM you provision:

```
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 14
pm2 set pm2-logrotate:compress true
pm2 set pm2-logrotate:rotateInterval '0 0 * * *'
pm2 save
```

This policy keeps **max 14 compressed files per log stream**, rotate when **10 MB** or **daily at 00:00 UTC**. Fits comfortably inside 2 GB free.

5.2.4 Recommended future aggregation (not in place today)

Ship logs to a log SaaS by adding one of:

- [Vector.dev](#) agent → tails PM2 → forwards to Datadog / New Relic / Grafana Cloud / OpenObserve
- Or PM2-logrotate-s3 variant → uploads `.gz` nightly to private S3 bucket
- Or Grafana Agent (LGTM stack)

Until then, when investigating issues without central log history you are **limited to last 14 compressed logs on disk** per §5.2.3 policy.

5.3 Alerting Thresholds

No alerts are wired yet. These are the thresholds the platform/DevOps team should feed into the first uptime + monitoring SaaS they adopt. Choose any: [Better Stack](#), [UptimeRobot](#) free tier for HTTP pings, then optionally add Prometheus node_exporter + Grafana Cloud metrics when budget permits.

5.3.1 Uptime / HTTP level

Alert	Source	Threshold	Severity (P1/P2/P3)	Snooze/suppression
Homepage HTTPS down	Uptime SaaS GET https://warm-hello.com/ 302 or 200	HTTP non-200/302 for 2 consecutive 60 s intervals	P1 — customer-facing outage	During deploy windows (±10 min around each pm2 restart)
Health endpoint 5-in-5 red	Uptime SaaS GET /api/health + body match <code>"ok":true</code>	not ok 5/5 1-min intervals	P1	During deploy
SSL cert expiry	Uptime SaaS TLS expiry monitor	≤ 10 days until expiry	P2 → P1 at ≤3 days	After renewal confirmed
App route p95 latency	Synthetic or RUM	p95 GET /dashboard > 3.0 s for 10 min rolling	P2	

5.3.2 Server OS metrics (Lightsail micro sensitive)

Monitor via [node_exporter](#) + Grafana / CloudWatch agent or vendor integrations:

Alert	Metric	Threshold (sustained 5 min)	Severity
CPU burst credit depletion	CPUUtilization > 85% AND CPUCreditsUsage = 100% on Lightsail dashboard	10 minutes	P1 — throttled requests pile up
High memory pressure	MemUsedPercent > 90%	5 minutes	P1 — OOM-kill imminent (Next build or GC)
Swap thrashing	PSWPagesIn/s + PSWPagesOut/s > 100 pages/s	10 minutes	P2
Root disk space	DiskSpacePercentFree < 10% (≤ 2 GB free on 20 GB root)	now	P2 → P1 at < 3%

Alert	Metric	Threshold (sustained 5 min)	Severity
Root disk inode exhaustion	InodeFreePercent < 5%	now	P2
Zombie / unstopable processes	ProcessesStateZombies ≥ 1 and re-occurring after 5 min gap	5 min	P3

5.3.3 Application level (parse from PM2 logs)

Forward `warmhello-error-0.log` into a log SaaS that can pattern-match and raise alerts:

Pattern in error log	Trigger	Severity
<code>Invalid environment variables</code> OR <code>ZodError: \[</code>	Any line	P1 — app is crash-looping; JOB_SIGNING_SECRET < 32 most likely
<code>PrismaClientInitializationError</code>	≥ 3 occurrences in 10 min window	P1 — DB down or DATABASE_URL wrong
<code>P2002</code> (unique constraint) OR <code>P2025</code> (not found) spam on endpoints that should never fail e.g. <code>/api/account/delete</code>	≥ 10 errors in 30 min window	P2
<code>Cross-origin request blocked</code> NOT matching exempt routes (webhooks/jobs/magic)	≥ 20 occurrences / hour (not bot noise)	P2 investigate CORS misuse / attack
HTTP 429 bursts (rate limiter) from one IP	≥ 50 429s per IP per hour	P3 → P1 if pattern looks like credential stuffing
<code>/api/billing/checkout</code> returning 400 with Stripe auth errors	≥ 1 / hour during active business hours	P3
Stripe webhook <code>constructEvent FAILED</code>	≥ 5 per hour	P3 → P1 if 100% failure rate (webhook secret wrong or clock skew)
QSTASH verify signature failed	≥ 3 in 1 hour	P2 — someone forging <code>/api/jobs/*</code> calls

5.4 Performance dashboards

Start with a manual SSH dashboard script, migrate to Grafana later. Save as `/home/admin/bin/perf-dash.sh`:

```
#!/bin/bash
set -euo pipefail
echo "=== Warm-Hello Perf Dashboard $(date -u) ==="
echo "--- loadavg / mem / swap ---"
uptime ; free -h ; vmstat 1 2 | tail -1
echo "--- disk / inode ---"
df -h / ; echo ; df -i /
echo "--- PM2 status ---"
```

```
pm2 status warmhello --machine | jq -c '.processes[0].pm2_env.status, .processes[0].monit' 2>/dev/null ||
echo "--- netstat on 8080 ---"
ss -tan4 state established '( sport = :8080 )' | wc -l ; echo "established sockets to 8080"
echo "--- recent errors (tail 40 lines error log) ---"
tail -n 40 /home/admin/.pm2/logs/warmhello-error-0.log
```

5.5 Baseline numbers (reference for anomaly detection)

Establish on a healthy idle day → these are your "normal" baselines:

- Idle RSS of single warmhello PM2 process: ~180 → 240 MB (under 512 MB + 2 GB swap budget)
- Build time cold (full `next build`): ~4.5 → 6.5 minutes
- Next startup from `pm2 start` until routes ready for first request: ~9.7 → 12.3 seconds
- `GET /api/health` response p50 on loopback: 6–15 ms, p95 < 60 ms
- `GET /` homepage (SSR with App Router) p50 80 ms → 280 ms (depends on cache hit inside Next ISR or none)
- Bcrypt hash cost factor 12 measured on prod-ish CPU: 191 ms per password hash (signup/password change) — interactive budget target ≤ 500 ms met with 2.6x headroom.
- Typical peak concurrent requests before PgBouncer `connection_limit=15` shows queuing: ~12 concurrent DB-backed requests (leave 3 headroom for migrations/internal work).

Chapter 6 — Security, Access & Compliance

6.1 Access Control

6.1.1 Server access (SSH)

Access type	Role(s)	Mechanism	Notes
SSH to Lightsail host	Owner / Admin (currently 1 person)	Ed25519 SSH keypair + password login DISABLED (<code>/etc/ssh/sshd_config PasswordAuthentication no</code>)	Default Lightsail Debian image ships key only — KEEP THIS — do not re-enable passwords.
Sudo on host	Owner / Admin	User <code>admin</code> in sudo group (password-less or password-on-sudo per Lightsail policy)	No secondary admins today. Add new users via <code>useradd -m -s /bin/bash <newadmin> ; usermod -aG sudo <newadmin></code> + install their Ed25519 pubkey to <code>/home/<newadmin>/.ssh/authorized_keys</code>

Access type	Role(s)	Mechanism	Notes
MFA on SSH	NOT ENABLED today	Recommend: google-authenticator-libpam + TOTP on sudo (low-priority hardening; not required for ops handoff)	

Audit quarterly:

```
# Who can login via SSH keys
grep -vE '^#|^$' /home/admin/.ssh/authorized_keys | wc -l
# Who is in sudo group
getent group sudo
# Last 5 successful SSH logins + last failed attempts:
last -5 -a
lastb -5 -a
```

6.1.2 Database access

Who	Access	Mechanism
App server (Prisma ORM)	Read/write on ALL tables	Shared role postgres.mjcgmecafwhlzqglcmqj + password embedded in DATABASE_URL and DIRECT_URL
Manual ops team	Read + write, DDL	Supabase console → SQL Editor (web) OR DIRECT_URL <code>psql</code> CLI
Browser / end users	NO DIRECT DATABASE ACCESS	Supabase anon key + Supabase SDK entirely disabled / unused . All data access goes through app server endpoints with explicit row ownership checks via signed session cookies.

6.1.3 Admin panels

- **No back-office / Django-style admin / Retool / Metabase dashboard deployed today.** Supabase Console → Table editor substitutes for ad-hoc manual ticket fixes.
- Application account delete endpoint exists (user self-service [/api/account/delete](#)). Support/ops-initiated delete requires direct SQL update via Supabase console until an admin panel is added.

6.1.4 Access rotation schedule (SHOULD policy)

1. SSH keypairs: rotate when an admin leaves OR every 12 months.
2. Supabase DB password (re-generate in Supabase Console → Project Settings → Database → Reset password for [postgres.mjcgmecafwhlzqglcmqj](#) role): **once per year OR on personnel change**. After reset, update `.env` `DATABASE_URL` + `DIRECT_URL` password segment → rebuild → `pm2 restart warmhello --time`.
3. Stripe secret key: rotate every 12 months. Old key revoked in Stripe Dashboard → Developers → API keys 5 minutes after new key activated on both server side `.env` `STRIPE_SECRET_KEY` + any embedded test keys.
4. AWS SES SMTP credentials (IAM user [ses-smtp-prod](#)): delete and regenerate SMTP password every 12 months.

5. `JOB_SIGNING_SECRET`: treat as any other signing secret — rotate every 6 months minimum. **When you rotate, be aware:** all outstanding unsubscribe links older than 1h expire, all in-flight onboarding grant tokens expire (10 min anyway so acceptable 6-min downtime once you pm2 restart), all existing v2 session cookies will have their signature fail → users are logged out (acceptable once per 6 months; if NOT acceptable → implement secret rotation with 2 keys then switch — out of scope today).

6.2 SSL/TLS & Domain Setup

6.2.1 HTTPS today — Lightsail managed

Warm-Hello runs behind Lightsail's managed HTTPS termination by default.

- **Inbound connection:** Browser ■ Lightsail public front (TLS 1.2+) ■ HTTP to our VM loopback 8080.
- **HSTS header** is added by app middleware at [middleware.ts:91-93](file:///C:/Users/mario/Documents/trae_projects/WarmHello/warmhello-app/middleware.ts#L91-L93): `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload` (2 years).
- If you ever run on a non-HTTPS environment locally in `NODE_ENV=production` → middleware also injects HSTS. In local dev (http localhost) HSTS omitted.

6.2.2 Recommended production hardening — custom Certbot + nginx reverse proxy

When adopting a custom domain / naked www redirect / direct TLS:

1. Point A record at Lightsail public IPv4 per §1.3.4.
2. Install nginx + certbot:

```
sudo apt-get install -y nginx certbot python3-certbot-nginx
```

3. Drop nginx config `/etc/nginx/sites-available/warmhello`:

```
server {
    server_name warm-hello.com www.warm-hello.com;

    # Security headers that nginx sets BEFORE app sees request (redundant with middleware but safe)
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    # Reverse proxy to Next.js loopback only (8080 NOT internet-reachable)
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
        proxy_send_timeout 60s;
        client_max_body_size 12m;
    }

    # ACME HTTP-01 challenge pass-through for certbot
    location /.well-known/acme-challenge/ {
```

```

    root /var/www/html;
}
}

```

4. Enable: `sudo ln -sf /etc/nginx/sites-available/warmhello /etc/nginx/sites-enabled/ ; sudo nginx -t && sudo systemctl reload nginx`

5. Issue certificate + auto-renew:

```

sudo certbot --nginx -d warm-hello.com -d www.warm-hello.com --redirect --hsts --uir --agree-tos -m admin@warm-hello.com
# certbot installs a systemd timer that auto-renews 30 days before expiry

```

6. Validate auto-renew: `sudo certbot renew --dry-run`

6.2.3 TLS protocols / ciphers (modern profile)

Certbot nginx auto-enforces TLS 1.2+, disables weak RSA-3DES suites. After install confirm with:

```

nmap --script ssl-enum-ciphers -p 443 warm-hello.com | grep -E "TLSv|Negotiation"

```

Acceptable → only **TLSv1.2** and **TLSv1.3** present, no **TLSv1.0** / **TLSv1.1**.

6.3 Compliance & Data Privacy

6.3.1 Personal Data Inventory (what the app stores, data classification)

Table / Column	Data category	PII?	Retention	Deletion mechanism
<code>subscribers.email</code>	Contact email	■ Yes (EU GDPR art 4)	Until user account deleted OR 7y since last login without explicit opt-in marketing consent	<code>POST /api/account/delete</code> sets <code>deletedAt</code> + nulls; final purge after 30 days manual SQL run OR Supabase batch delete job
<code>subscribers.phoneNumber</code>	Mobile phone for SMS check-ins	■ Yes	Same as email above	Same delete endpoint nulls column
<code>subscribers.firstName / lastName / birthday</code>	Personal identifiers	■ Yes	Same	Same
<code>subscribers.address / city / postalCode / province / country</code>	Home address (optional, for alternate dispatch)	■ Yes	Same	Same
<code>households.name</code>	Household display name (often PII-adjacent, e.g. "Altamira Family")	■■ Adjacent	Same	Cascaded to household deletion
<code>checkins.checkinAt / token / respondedAt / response</code>	Check-in response data, response can contain free-text "message"	■■ Possibly PII if user writes PII (designed not to)	2 years after user account deletion OR 7y from creation, whichever is shorter	<code>prisma.checkin.deleteMany({where:{subscriber:{email:...}}})</code>

Table / Column	Data category	PII?	Retention	Deletion mechanism
<code>subscribers.passwordHash</code>	Bcrypt hash v12, NEVER PLAINTEXT	■■ Credential -derived	Same as subscriber lifetime	Delete subscriber removes hash
<code>subscribers.passwordResetTokenHash</code>	Bcrypt 1-way hash of reset token	■■ Credential -derived	TTL 1 hour at app level; row-level tokenExpiresAt expires; garbage cleanup SQL monthly prunes	Update NULL after token use or expiry
<code>authMagic</code> rows (future)	Magic link tokens (if path ever restored)	■■ Credential -derived	15-minute absolute TTL	Prisma deleteMany past expiration
<code>smsMessages</code> / <code>emailsSent</code> tables (if added)	Message logs	■ Possibly PII	12 months after send then purge	Nightly batch SQL

6.3.2 Encryption at rest

- **Database (Supabase):** AES-256 at rest enabled automatically by Supabase on all paid tiers; free tier too per AWS EBS encryption default under-the-hood.
- **Server disk:** AWS Lightsail root EBS volumes **encrypted by default** since 2023. Confirm via `aws lightsail get-instances --query 'instances[].{name:name,encryption:addOns[?name=="Auto Snapshot"].enabled}'` or console → Storage → Encrypted = Yes. If No → migrate plan by snapshot-restore onto an encrypted volume (mandatory for compliance).
- **Backups:** Supabase automated snapshots inherit at-rest encryption. If you take `pg_dump` locally per §4.3.2, encrypt them **before** uploading to 3rd party storage:

```
age -r AGE_PUBKEY_YOURTEAM &lt; warmhello-prod-backup-...dump &gt; warmhello-prod-backup-...dump.age
```
- **Secret keys on disk:** `.env` `chmod 0600` — only. Not filesystem-encrypted separately (no LUKS / eCryptfs / fscrypt today; acceptable on single-owner single-VM, document gap).

6.3.3 Encryption in transit

- Browser ■ server: **TLS 1.2+**, HSTS 2yr pinned
- Server ■ Supabase Postgres: TLS `sslmode=require` by Prisma, port 5432
- Server ■ Stripe/Telnyx/Qstash: HTTPS, TLS 1.2+, cert chain validated
- Server ■ SES SMTP: TLS wrapper on **port 465** (not STARTTLS 587; wrapper is mandatory — no cleartext handshakes)
- Session cookies: `HttpOnly`; `SameSite=Lax`; `Secure` flags when served HTTPS; `maxAge=7 days`. Cookie payload contains ONLY `subscriberId|issuedAtEpochMs` HMAC signature (no name/email/PII baked in).
- Passwords: `bcryptjs v3`, `cost=12`, unique salt per row, never visible in logs or API responses.

6.3.4 Laws & frameworks alignment (current status — "best effort, not audited")

- **GDPR (EU):** App supports:

- Right of access (self-service dashboard shows account data → export JSON endpoint not built yet; ops manual SQL dump via Supabase console today)
- Right to be forgotten → `POST /api/account/delete` endpoint sets `subscriber.deletedAt` + checkins anonymize
- Unsubscribe of marketing email via `/unsubscribe/[token]` (GET → confirmation page → POST mutates state, 1h CSRF signed token)
- Legal pages deployed: `/privacy`, `/terms`, `/data-processing-agreement`
- **CAN-SPAM / CASL (SMS + email)**: Every marketing email has clear List-Unsubscribe header + one-click unsubscribe link (uses short-TTL tokens). Telnyx 10DLC SMS: STOP reply to opt-out handled by Telnyx profile auto-opt-out + sync to `subscriber.subscribedAt = NULL` via Telnyx webhook once added.
- **HIPAA / PHI**: App does NOT process PHI today (no insurance, diagnosis, prescription fields). If the roadmap ever adds medical-alert integration data flows → sign BAA with Supabase + Stripe + AWS first. Not applicable to today's system.
- **PCI DSS**: Payments flow via Stripe Checkout Sessions (hosted pages on stripe.com domain, never WARMHELLO server or browser JS touches card PAN). No card storage on servers, no PCI SAQ A-EP burden, falls under **SAQ A** level (easier). Maintain Stripe PCI attestation via Stripe compliance console; nothing local to Warm-Hello server beyond storing Stripe API keys.

6.3.5 Data breach response (§7.6 Troubleshooting also references)

If `.env` file exposure, database backup leak, or SSH key compromise is suspected:

1. Isolate instance: immediately add UFW deny-in rule to all but SSH admin IP OR stop PM2 process.
2. Rotate ALL 11 secret keys from §3.1.1 list — batch, in parallel via provider consoles.
3. Notify subscribers (legal: `privacy@warm-hello.com` + legal counsel) within statutory notification windows: - GDPR: ≤ 72 hours after discovery to Supervisory Authority - CCPA / US state breach laws vary, most 30–45 days to residents - PCI: if PAN ever exposed beyond hosted checkout → notify Stripe within 24 h
4. Log every step, preserve system state (snapshot Lightsail) for forensics, rebuild from clean backups.

Chapter 7 — Troubleshooting Runbooks

Before any deep investigation, start with the three-step triage. It resolves ~70% of pages in <10 minutes.

Three-step triage (always run first):

1. `pm2 status warmhello` → status = online? uptime reasonable? CPU% > 90%? Mem% nearing 512 MB?
2. `curl -sS http://127.0.0.1:8080/api/health` → 200? integrations all true?
3. Last 80 lines of PM2 error log: `tail -n 80 /home/admin/.pm2/logs/warmhello-error-0.log | grep -v '^$'` — is it a recurring error?

If any step fails, go directly to the matching numbered runbook below.

7.1 Runbook 1 — Next Build OOM SIGABRT on Deploy (exit code 134, 512 MB micro)

Symptoms

- `next build` prints red `SIGABRT: abort` or `JavaScript heap out of memory` and a 50 MB stack dump to `stderr`
- Exit code 134 (bash convention: 128 + SIGABRT 6)
- Build grid of Static/Server/Static routes NEVER prints

Root cause (confirmed known)

V8 default heap sizing on a 512 MB host thinks it has ~380 MB; Next build's Terser minification + webpack chunk split peaks around ~900 MB virtual; OOM-killer or V8 internal abort triggers.

Fix (100% success — deploy script already uses it)

```
NODE_OPTIONS="--max-old-space-size=2048" pnpm exec next build
```

Why this works even with only 512 MB physical: V8 asks OS for 2 GB heap max; 2 GB swapfile absorbs ~1.2 GB cold pages, only hot working set (~300 → 450 MB) lives in RAM. Latency during minification slows down but still ≤ 6 min total build.

Permanent guardrail

Always wrap these in `NODE_OPTIONS="--max-old-space-size=2048"` on prod micro:

- `pnpm install --frozen-lockfile` (large dependency trees)
- `next build`
- `next start`
- `prisma generate` (optional; harmless to add)
- `tsc --noEmit` (optional once project >2k ts files)

False negative check

If still failing after `NODE_OPTIONS` → run `free -h` and confirm Swap total says 2.0 Gi; if swap missing, re-enable it per §2.2 Step 1 then retry build. Swap off + 512 MB = guaranteed failure.

7.2 Runbook 2 — PM2 started, curl to /api/health fails "curl(7) Failed to connect: Connection refused" for first minute

Symptoms

After `pm2 start` + immediate curl (within 15 seconds):

```
curl: (7) Failed to connect to 127.0.0.1 port 8080 after 0 ms: Couldn't connect to server
```

Root cause (confirmed known — Next.js cold start latency)

`next start` on this app takes **9.7 → 12.3 seconds** before it binds port 8080 and serves first request. During that window, PM2 reports "online" (because the process spawned) but the Node HTTP server is NOT yet listening. The deploy script already uses `sleep 25` then 5 retries to absorb this.

Remediation

Always wait at least 20 s before first health curl after `pm2 start` OR `pm2 restart`.

Deploy-mandatory sleep (copy this, never remove the 25 s):

```
pm2 restart warmhello --time
sleep 25
for i in 1 2 3 4 5; do curl -sS http://127.0.0.1:8080/api/health ; echo " (attempt $i)" ; sleep 5; done
```

If curl7 still fails at attempt 5

Process truly died. Check:

1. `pm2 status warmhello` → is status "errored" or "stopped" not "online"?
2. `tail -n 120 /home/admin/.pm2/logs/warmhello-error-0.log` — look for `Invalid environment variables: JOB_SIGNING_SECRET Expected string length ≥ 32` (this is #1 post-deploy crash cause now).

7.3 Runbook 3 — App crash-loop immediately after pm2 start (Zod env parse fails)

Symptoms

- `pm2 status` shows Warm-Hello restarts every 3-10 s, uptime never >20 s
- Error log tail contains line: `Invalid environment variables:` followed by a key name

Common offending keys today (ordered by likelihood)

1. `JOB_SIGNING_SECRET` — **Expected string, length ≥ 32** - `lib/env.ts line 59` enforced this in security hardening commit - Old `.env` had a short dev-only 16 char secret → app will **NOT** boot with it
2. `DATABASE_URL` / `DIRECT_URL` — blank on a deployment that needs DB
3. `SMTP_PORT` — non-numeric / negative
4. `APP_URL` / `NEXT_PUBLIC_APP_URL` / `SHORT_LINK_BASE_URL` — invalid URL format (forgot `https://` prefix)

Fix step-by-step

1. Print the offending error: ```bash grep -A 20 "Invalid environment variables" /home/admin/.pm2/logs/warmhello-error-0.log | head -n 40 ```
2. Edit `.env` and fix the key. For `JOB_SIGNING_SECRET < 32` → generate new entropy: ```bash node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))" ```
3. Save + `chmod 0600 .env`
4. **Rebuild** (`NEXT_PUBLIC_*` vars baked into JS bundle) then restart: ```bash NODE_OPTIONS="--max-old-space-size=2048" pnpm exec next build pm2 restart warmhello`

```
--time sleep 25 ; curl -sS http://127.0.0.1:8080/api/health ``
```

If you need to fall back to older code that allowed short JOB_SIGNING_SECRET

→ See §2.5 Rollback A, roll back before security pass. Not recommended.

7.4 Runbook 4 — Prisma "P1001 Can't reach database server at X.supabase.com:5432"

Symptoms

- Any DB-touch endpoint returns 500
- PM2 error log tail 30 lines has `PrismaClientInitializationError: error: Error validating datasource \db\: the URL must start with the protocol \postgresql://\ or \postgres://\.` → no OR P1001: Can't reach database server

Step-by-step

1. **Test network-level reachability FROM SSH server shell** `bash`
`HOST="aws-0-ca-central-1.pooler.supabase.com" PORT=5432 timeout 5 bash -c "cat < /dev/null > /dev/tcp/$HOST/$PORT" 2>/dev/null && echo "PORT $PORT OPEN to $HOST" || echo "FAIL - DB TCP NOT reachable" ` If FAIL → 99% an outbound firewall rule, UFW, or AWS VPC security group egress deny on :5432. - Temporarily disable UFW to test: sudo ufw disable` → if it starts working → UFW rule order issue (default outgoing allow MUST be enabled at top; see §2.2 Step 4).`
2. **Check DB role password correctness** - Paste `DIRECT_URL` into **Supabase SQL Editor on console** web page — does SQL Editor work? If web also fails → project is paused on Supabase free tier after 7 days idle! Restore it on Supabase Console Project → Resume. - If web works but URL fails → reset password for `postgres.mjcgmecafwhlzqglcmqj` from Console → Project → Settings → Database → Reset password. Update `.env` then rebuild + restart.
3. **PgBouncer mode mismatch** — ensure pooled URL has `?pgbouncer=true&connection_limit=15` query string after `/postgres` db name. The direct URL MUST NOT have `pgbouncer=true`.
4. **Verify .env URL has NO whitespace** — very common paste error: `bash node -e "const u=require('fs').readFileSync('.env','utf8').match(/^DATABASE_URL=(.*)$/m)||[])[1]||''; console.log('len=', u.length, 'startsWith=', u.startsWith('postgres://'), 'hasSpace=', /\s/.test(u));" ` Correct result: startsOk=true hasSpace=false`.`

7.5 Runbook 5 — Too Many 429s on endpoints (rate limiter blocking legitimate users)

Symptoms

- Users see toast "Too many requests. Try again in X minutes."
- Response body from endpoint returns HTTP 429 with header `Retry-After: <seconds>`

- PM2 out log shows security audit kind names `_RATE_LIMITED`

Diagnosis

Identify key dimension → run PM2 error log grep:

```
grep -h "RATE_LIMITED\|429" /home/admin/.pm2/logs/warmhello-out-0.log /home/admin/.pm2/logs/warmhello-error-0
```

Common dimension keys: `ip:`, `subscriberId:`, `email:`, `householdId:`, `token:`.

Remediation options

1. **Single user blocked (genuine legitimate burst):** Restart the process → this wipes ALL in-memory rate limit buckets clean. Drawback: also gives attackers fresh buckets, so only do for genuine known internal single-user case. `bash pm2 restart warmhello --time ```
2. **Legitimate bot (monitoring service / health checker / uptime SaaS):** Add the source IPv4 to a new trusted IP bypass list in `lib/rate-limit.ts` (short-circuit before `checkRateLimit`).
3. **Attack credential stuffing → real 429s are GOOD:** The rate limiter is functioning correctly. Upgrade to a Redis-backed rate limiter + permanent IP block list to continue scaling defense.
4. **Limits too low for a known busy day (launch, promo):** Adjust constants up in each route file's `checkRateLimit(key, windowMs, limit)` call. Typical constants to double: - Subscriber create signup `3/IP/5min → 10/IP/5min` - Timeline actions that users legitimately click several times in a row.

Architectural note on rate limiter (CRITICAL for future)

§1.4 caveat restated: rate limiter state is in-process only. If we ever scale to 2 Node processes via `pm2 -i max` or a second Lightsail replica, state diverges between processes. Fix by swapping `Map<string, number[]>` bucket to **Upstash Redis** (Lua script sliding-window pattern).

7.6 Runbook 6 — PgBouncer / Connection Pool Exhaustion (all endpoints slow, 504 or no response)

Symptoms

- Every DB-backed endpoint p90 goes from <400ms → >3s for ≥10 min continuous
- PM2 error tail shows repeating `Timed out during fetch in 10000 ms` or `PrismaClientKnownRequestError: P2024: Timed out fetching a new connection from the connection pool`
- Active DB sessions in Supabase Console → Server → Connections shows 15/15 connections (maxed out)

Step-by-step

1. Identify stuck idle queries / abandoned transactions: `bash psql "$DIRECT_URL" -c " SELECT pid, now()-query_start AS dur, state, wait_event_type, wait_event, left(query, 120) query FROM pg_stat_activity WHERE state <> 'idle' OR wait_event IS NOT NULL ORDER BY query_start ASC;" ```

2. For any session `dur > 5 min` state `idle in transaction` → kill it: ```sql SELECT pg_terminate_backend(<pid_from_above>); ```
3. Temporary 30-min reprieve if we can't deploy: ```bash pm2 restart warmhello --time # drops app-side pool and re-initializes 15 fresh ```
4. Long-term fix: - Add Prisma `pool_timeout` explicit tuning in schema `datasource url = env("DATABASE_URL")` → add `?pool_timeout=5` (fail fast) - Bump `connection_limit` from 15 → 30 on URL, and confirm PgBouncer pooler on Supabase plan supports ≥ 30 (Pro plan minimum)

7.7 Runbook 7 — Stripe Webhook 400 "constructEvent FAILED" 100% of callbacks

Symptoms

- Stripe Dashboard → Webhooks → Warm-Hello webhook endpoint → Recent deliveries every call = 400 "Webhook Error: ..."
- PM2 error log has lines with substring `[stripe-webhook:verify-signature] constructEvent FAILED`

Root causes (sorted by likelihood)

1. `STRIPE_WEBHOOK_SECRET` env var is `whsec_` from **WRONG Stripe webhook endpoint** (you created 2 endpoints and pasted the wrong one).
2. **Server clock skew** — Stripe signatures include a `t=` timestamp; 5 min tolerance is default. Run `date -u` on Lightsail SSH and compare to `date -u` on your laptop; drift > 60 s → configure NTP: ```bash sudo timedatectl set-ntp true ; timedatectl status ```
3. **Reverse proxy / body parser double-read**: Next.js route already consumes request body stream as raw; ensure no middleware touched it and rewrote bytes. (Never an issue today since we use standard `NextRequest.text()` pattern — keep that pattern.)

Verification step after fix

```
# From Stripe Dashboard → Webhook → Send test event:
# type = customer.updated, expected status = 200 + JSON {"received":true}
# OR from CLI if Stripe CLI installed:
stripe listen --forward-to https://warm-hello.com/api/webhooks/stripe
```

7.8 Runbook 8 — 403 "Cross-origin request blocked" for internal POSTs (CSRF middleware firing false-positive)

Symptoms

- A working user form suddenly POST → 403 JSON `{"ok":false,"error":"Cross-origin request blocked."}`
- The route is NOT on the whitelist: `/api/webhooks/*`, `/api/jobs/*`, `/api/auth/magic`

- Hitting the endpoint with `curl -X POST http://127.0.0.1:8080/api/<route>` also fails with 403 (because curl sends no Origin header by default)

Diagnosis & quick fix for admin testing

1. Admin `curl` 403s are expected behavior. Correct curl to include Origin header from site whitelist:

```
bash
curl -sS -X POST http://127.0.0.1:8080/api/auth/change-password \ -H "Origin: https://warm-hello.com" \ -H "Content-Type: application/json" \ -d '{ "oldPassword": "x", "newPassword": "yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy" }'
```
2. Real user browsers are failing → check the `NEXT_PUBLIC_APP_URL` env var exactly matches browser URL origin (scheme+host+port). Common mistake: `https://warm-hello.com` in browser but `APP_URL=http://warm-hello.com` in `.env` (no https) — origin comparison fails because protocol differs. Fix in `.env`, rebuild, restart.

If an internal cron / third-party caller legitimately POSTs with no origin

Add the path prefix to `EXTERNAL_POST_WHITELIST_PREFIXES` in [middleware.ts:14-18](file:///C:/Users/mario/Documents/trae_projects/WarmHello/warmhello-app/middleware.ts#L14-L18). Re-deploy.

7.9 Disaster Recovery — Formal RPO / RTO + Catastrophic Outage Protocol

Definitions

- **RPO (Recovery Point Objective):** Max acceptable data loss during disaster = ≤ 24 hours (Supabase daily snapshot worst-case). Supabase Pro 7-day PITR → RPO ≤ 5 minutes achievable.
- **RTO (Recovery Time Objective):** Max downtime until back online = ≤ 60 minutes with single operator. Requires operator SSH shell access + Supabase console access.

Step-by-step Disaster Recovery (data-center-level failure / ransomware / stolen SSH key → full rebuild)

1. **0 → 5 minutes: Declare disaster** — Stop further writes. Add TXT `_dmarc` temp to quarantine or suspend QStash schedules via Upstash Console. Stop PM2: `pm2 stop warmhello` or (if host down) assume host is lost.
2. **5 → 15 min: Choose restore target** - For PITR eligible (Pro): Supabase Console → Backups → PITR → pick timestamp 5 minutes BEFORE disaster started - For daily snapshots only: click latest green snapshot before disaster - **In parallel on a new Lightsail instance** — provision a new Debian 12 micro per §2.2 Steps 1–6. Skip DB bootstrap steps until Supabase restore done.
3. **15 → 40 min: Spin up new VM + DB swap** - DB: Wait for Supabase restore operation to report Complete (UI). Credentials unchanged in majority of cases. - VM: Follow §2.2 step 7 onwards. Copy the **rotated (new)** `.env` to `/home/admin/warmhello/warmhello-app/warmhello-app/.env`. - Build + start; `sleep 25`; health check.
4. **40 → 55 min: Cut over DNS / TLS** - DNS A record `warm-hello.com` → new VM public IPv4 - Reissue certbot cert if custom TLS (recommended): §6.2.2 steps - Monitor first 1 000 live requests via error log tail, integrations all five true

5. **55 → 60 min: Wrap** - QStash schedules re-enabled - Stripe webhook endpoint updated in Stripe Dashboard Developers → Webhooks if endpoint URL changed - Telnix SMS webhook URL updated if changed - Postmortem: update docs, file ticket against root cause, verify nightly backup runs.

7.10 Escalation matrix

Formal escalation path & vendor contact directory has been broken into its own document: [\[emergency-contacts.md\]\(emergency-contacts.md\)](#). Keep that file printed / pinned next to this runbook during a P1 incident.

Chapter 8 — Emergency Contacts & Escalation

Keep this page open during any P1 incident alongside §7.9 Disaster Recovery. Call / escalate in this order.

8.1 Internal Escalation Path — Warm-Hello

Step	Role	Name	Contact (phone / email)	Authority	Response SLA
1	Incident Commander / Primary on-call	PLACEHOLDER_PRIMARY_ONCALL_NAME	■ PLACEHOLDER_PHONE_PRIMARY / ✉ PLACEHOLDER_EMAIL_PRIMARY	All decisions, final word	15 min
2	Secondary on-call backup	PLACEHOLDER_SECONDARY_ONCALL_NAME	■ PLACEHOLDER_PHONE_SECONDARY / ✉ PLACEHOLDER_EMAIL_SECONDARY	Fill if primary unreachable	25 min
3	Owner / Executive signoff	PLACEHOLDER_OWNER_NAME	■ PLACEHOLDER_PHONE_OWNER	Legal communications, GDPR breach notice signoff, spend authorization for new VMs/licenses	1 h

8.2 Third-Party Vendor Support Channels

Each vendor's severity is mapped to Warm-Hello P1/P2/P3. Open a ticket with **exact** ticket severity = matching column; vendors have SLA response windows per plan.

Vendor	What they own for us	Support Portal URL	Severity to open ticket	Known account / plan IDs (fill placeholders)
AWS Lightsail	VM, attached block storage, VPC firewall, Static IP, Snapshot backups	https://support.console.aws.amazon.com/support/home (Business plan required for < 1 h P1; free tier only forum)	VM unreachable / attach volume failure → P1 Business	Account ID PLACEHOLDER_AWS_ACCOUNT_ID. Region PLACEHOLDER_AWS_REGION. Lightsail instance name PLACEHOLDER_LIGHTSAIL_INSTANCE_NAME.
AWS SES	SMTP TLS relay, IAM SMTP users, DKIM/SPF signing, deliverability & bounce feedback loops	Same AWS support above → Create case → Service: Simple Email Service	Bounce rate > 10% on a campaign, or "550 Mailbox unavailable" for known-good recipients → P2	Identity ARN: PLACEHOLDER_SES_IDENTITY_ARN. Region for SES: PLACEHOLDER_SES_REGION. SMTP username AKIA... last 4 digits PLACEHOLDER_SMTP_USER_LAST4.
Supabase	PostgreSQL hosting, PgBouncer, PITR backups, project auto-pause on free tier	https://supabase.com/dashboard/support — Pro + Team plans have email SLA.	DB project in "Paused" state and cannot resume / restore failing / PITR restore hung → P1 Team/P1 Enterprise	Project Ref = mjcgmecafwhlzqglcmqj (this is the string embedded in role postgres.mjcgmecafwhlzqglcmqj). Org = PLACEHOLDER_SUPABASE_ORG_NAME.
Stripe	Billing, Checkout Sessions, Subscription state, Webhook signing	https://support.stripe.com/	Billing outage (Checkout Session creation 100% failure rate with Stripe-side 5xx) → P1 Enterprise/Premium. Webhook signature 100% failure on app-side we handle first then escalate only if all docs checked.	Account ID (acct...): PLACEHOLDER_STRIPE_ACCT_ID. Live mode publishable key prefix pk_live... : PLACEHOLDER_STRIPE_PUBLISHABLE_KEY_PREFIX.
Telnyx	10DLC SMS, 10DLC campaign registration, SMS webhooks, long-code numbers	https://portal.telnyx.com/#/support/tickets	10DLC campaign approval delayed past T+14 biz days / SMS delivery ratio < 50% for ≥1h → P2 Pro	Messaging Profile ID: PLACEHOLDER_TELNYX_MSG_PROFILE_ID. Number order DID: PLACEHOLDER_TELNYX_FROM_DID. Connection Webhook ID: PLACEHOLDER_TELNYX_CONNECTION_ID.
Upstash (QStash)	Scheduled jobs and queue fan-out → calls our /api/jobs/* routes	https://console.upstash.com/support	QStash 429 rate limit on our token, or 100% delivery error rate → P2 Pro	QStash namespace (copy from console): PLACEHOLDER_UPSTASH_NAMESPACE. Last 6 chars of token stored in .env (NEVER full): PLACEHOLDER_QSTASH_TOKEN_LAST6.

Vendor	What they own for us	Support Portal URL	Severity to open ticket	Known account / plan IDs (fill placeholders)
Domain Registrar	warm-hello.com domain ownership, DNSSEC, nameserver glue records if self-hosted DNS	Registrars vary: GoDaddy → godaddy.com/help ; Namecheap → namecheap.com/support ; AWS Route53 → same AWS account case as Lightsail	Nameserver change not propagating in 48 h, DNSSEC bogus, domain about to expire and auto-renew didn't run → P2	Registrar name: PLACEHOLDER_DOMAIN_REGISTRAR . Registrant email on file: PLACEHOLDER_REGISTRANT_EMAIL. Expiry date (YYYY-MM-DD): PLACEHOLDER_DOMAIN_EXPIRY_DATE. Auth code (EPP): OFFLINE ONLY — stored in physical safe / 1Password vault, NOT in this doc.
CDN / WAF (future if Cloudflare / Fastly adopted)	TLS, WAF rules, cache eviction, DDoS mitigation upstream	Cloudflare support	DDoS layer-3/4 saturating the single micro instance, HTTPS cert issuance failing → P1 Business/ENT	Zone ID: PLACEHOLDER_CDN_ZONE_ID. Account: PLACEHOLDER_CDN_ACCOUNT_EMAIL.

8.3 Legal / Compliance / Breach Notification Contacts

Role	Name / Firm	Contact	Trigger
Data Protection Officer (EU GDPR rep if no in-house)	PLACEHOLDER_DPO_NAME	✉ PLACEHOLDER_DPO_EMAIL	User personal data breach → within 72h notify Supervisory Authority (Ireland/UK/DE where users live)
Outside Privacy & Corporate Counsel	PLACEHOLDER_LEGAL_FIRM	✉ PLACEHOLDER_LEGAL_EMAIL ■ PLACEHOLDER_LEGAL_PHONE	Breach affecting ≥ 500 EU users, or regulatory inquiry, or PCI audit subpoena
Insurance (Cyber / E&O if policy in place)	PLACEHOLDER_INSURANCE_CARRIER	Claim # PLACEHOLDER_CYBER_CLAIM_PHONE	Ransomware incident, PCI forensic investigation required, or any incident > \$10k damages estimate
PR / Communications lead	PLACEHOLDER_PR_CONTACT	✉ PLACEHOLDER_PR_EMAIL	Press inquiry about a breach or long outage

8.4 P1 Incident Call Bridge Information

Keep pre-created and ready — do not fumble with setup during P1:

- **Zoom / Teams link:** PLACEHOLDER_INCIDENT_VIDEO_LINK
- **Bridge dial-in (US):** PLACEHOLDER_PHONE_BRIDGE (access code PLACEHOLDER_BRIDGE_PIN)
- **Shared live incident notes doc (Google Docs / Notion):** PLACEHOLDER_INCIDENT_NOTES_LINK

- **Public status page (Better Uptime / Instatus):** `PLACEHOLDER_PUBLIC_STATUS_PAGE_URL` (OPTIONAL — if none, use pinned @warmhelloc Twitter/X or LinkedIn account + update every 30 min during active outage)

8.5 Open a P1 incident checklist

Copy paste this list into the shared incident notes doc. Check off within first 15 min of declaration:

- ☐ Declare severity **P1** on video bridge, assign single Incident Commander.
- ☐ Scribe appointed (someone separate from IC writes timestamped notes).
- ☐ Host affected identified: `hostname -a` on lightsail + Supabase project status page captured (screenshots saved).
- ☐ Timeline: **When did it start happening? When was it first reported?** (minutes matter for PITR recovery target)
- ☐ Impact statement drafted: 1-sentence what's broken, % of users affected, ETA to preliminary fix.
- ☐ Communications: first 30-min-window status posted publicly (status page + socials) within 30 min of P1 declaration, updated every 30 min until resolved, final "resolved + postmortem forthcoming" post.
- ☐ Vendor tickets OPENED in parallel for each suspected vendor (DO NOT wait to open them sequentially; open all within 10 min).
- ☐ Snapshot VM disk + Supabase DB before any rollback attempt (forensics if later breach).
- ☐ Once fix applied, 15 min soak of live traffic, then close P1 → downgrade to P2 → P3 over following 24h.
- ☐ Within 3 business days → full Root Cause Analysis document + action items assignees. Share with leadership.

8.6 Placeholder Fill-In Table — Once Before Go-Live

Before you consider documentation complete, sit down with a text editor and replace every `PLACEHOLDER_*` token in the entire `documentation/` folder with actual values. The list below enumerates the tokens:

Token name	Where it appears	How to get the real value	
<code>PLACEHOLDER_LIGHTSAIL_HOSTNAME</code>	\$01 architecture	<code>hostname -f</code> in SSH	
<code>PLACEHOLDER_LIGHTSAIL_PUBLIC_IPV4</code>	\$01, 02, 06, emergency contacts	<code>curl -4 ifconfig.co</code>	
<code>PLACEHOLDER_LIGHTSAIL_PRIVATE_IPV4</code>	\$01	<code>`hostname -I \</code>	<code>awk '{print</code>
<code>PLACEHOLDER_REGION</code>	\$01 architecture	Lightsail Console → Instance → Region line	
<code>PLACEHOLDER_MX_PROVIDER</code>	\$01 DNS	Google Workspace / Fastmail / Microsoft 365 — pick one	

Token name	Where it appears	How to get the real value	
PLACEHOLDER_DKIM_SELECTOR	\$01 DNS	AWS SES → Verified Identities → DKIM section prints the 3 selector CNAMEs to create	
PLACEHOLDER_SUPABASE_POOLER_PASSWORD	\$03 env matrix production block	Supabase → Project Settings → Database → Pooler connection string	
PLACEHOLDER_SUPABASE_DIRECT_PASSWORD	\$03 env matrix production block	Same → Database → Session mode direct connection string	
PLACEHOLDER_JOB_SIGNING_SECRET_MIN_32_CHARS_GENERATED_VIA_NODE_ABOVE	\$03 env	<code>node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"</code>	
PLACEHOLDER_TELNYX_WEBHOOK_SECRET	\$03 env	Telnyx portal → Connections → your webhook → Signing Secret	
PLACEHOLDER_UPSTASH_QSTASH_TOKEN_PROD	\$03 env	Upstash Console → QStash → Project → API Keys tab	
PLACEHOLDER_ENCRYPTED_BACKUPS_BUCKET	\$04 manual backup command	Create S3 bucket → copy bucket name	
PLACEHOLDER_PRIMARY_ONCALL_NAME / PHONE / EMAIL	\$8.1	Your contact details + backup	
PLACEHOLDER_AWS_ACCOUNT_ID	\$8.2 vendor support	<code>aws sts get-caller-identity --query 'Account' --output text</code>	
PLACEHOLDER_AWS_REGION	\$8.2 vendor support	Console top-right us-east-1 etc	
PLACEHOLDER_LIGHTSAIL_INSTANCE_NAME	\$8.2 vendor support	Lightsail Console → Instances list	
PLACEHOLDER_SES_IDENTITY_ARN	\$8.2 vendor support	SES → Verified identities → click domain → ARN at top	
PLACEHOLDER_SES_REGION	\$8.2 vendor support	SES → drop down region selector	
PLACEHOLDER_SMTP_USER_LAST4	\$8.2 vendor support	<code>.env SMTP_USERNAME=AKIAxxxx</code> copy the last 4 chars of the 20 char key	
PLACEHOLDER_SUPABASE_ORG_NAME	\$8.2 vendor support	Supabase → Org switcher → Org name	
PLACEHOLDER_STRIPE_ACCT_ID	\$8.2 vendor support	Stripe Dashboard → top right avatar → Profile → Account settings → Account ID starts <code>acct_</code>	
PLACEHOLDER_STRIPE_PUBLISHABLE_KEY_PREFIX	\$8.2 vendor support	Stripe → Developers → API keys → Publishable key → copy <code>pk_live_...</code> first 12 chars	
PLACEHOLDER_TELNYX_MSG_PROFILE_ID	\$8.2 vendor support	Telnyx → Messaging → Messaging Profiles → click profile → URL path UUID	
PLACEHOLDER_TELNYX_FROM_DID	\$8.2 vendor support	<code>.env TELNYX_FROM_NUMBER=...</code>	

Token name	Where it appears	How to get the real value	
PLACEHOLDER_TELNYX_CONNECTION_ID	§8.2 vendor support	Telnyx Portal → Connections → your app → UUID in URL bar	
PLACEHOLDER_UPSTASH_NAMESPACE	§8.2 vendor support	Upstash Console → Namespace dropdown	
PLACEHOLDER_QSTASH_TOKEN_LAST6	§8.2 vendor support	.env QSTASH_TOKEN last 6 chars	
PLACEHOLDER_DOMAIN_REGISTRAR / PLACEHOLDER_REGISTRANT_EMAIL / PLACEHOLDER_DOMAIN_EXPIRY_DATE	§8.2 vendor support	WHOIS query + registrar account → Domains → list	
PLACEHOLDER_CDN_ZONE_ID / PLACEHOLDER_CDN_ACCOUNT_EMAIL	§8.2 vendor support	Populate only if Cloudflare/Fastly added later; leave as-is now	
Legal/PR/Insurance rows in §8.3	§8.3	Populate if/when you retain outside counsel; leave today as TODOs	
Video bridge / notes doc / status page	§8.4	Create once, leave URL links here	